Explica: Erlan Garcia (Rol 1)

Clasificador de Sentimiento de Reseñas en Español (NLP + ML)

1. El Problema Real del Dominio (Hoteles y Restaurantes)

- Los negocios turísticos reciben cientos de comentarios en Google Maps, TripAdvisor y Booking.
- Leerlos manualmente impide detectar a tiempo quejas graves: cobros indebidos en Bs, comida fría o mala higiene.
- Pregunta de decisión: ¿La reseña expresa satisfacción (Positivo) o insatisfacción (Negativo)?
- Objetivo: Construir el pipeline completo visto en clase desde el texto crudo hasta la tabla TF-IDF y el modelo.

2. Equipo y Responsabilidades en Git (4 Ramas)

- Integrante 1 — Erlan Garcia (@Erlan75): Generación del dataset (resenas.csv), EDA (Cuaderno 01) y módulo limpieza_texto.py.
- Integrante 2 — María Montaño (@M-2026-M): Experimentos NLP (Cuaderno 02), Escalera de Modelos y script entrenar.py.
- Integrante 3 — Ronny Choque (@DnaetFirst): Pipeline reproducible (.joblib), validaciones API y 10 pruebas pytest.
- Integrante 4 — Luis Miguel Nunez (@lmnunez): Matrices de confusión, pesos TF-IDF (Cuaderno 03), Ficha 9 Casilleros y Frontend.

Explica: Erlan Garcia (Rol 1)

Construcción del Dataset Balanceado y Ruido Realista en Español

¿Por qué 1.200 reseñas balanceadas (50% / 50%)?

- Archivo data/resenas.csv generado con semilla fija (semilla = 42) para que cada ejecución sea idéntica.
- Exactamente 600 reseñas positivas y 600 negativas (sin valores nulos ni duplicados).
- Regla vista en clase: Si entrenamos con clases desbalanceadas, el modelo memoriza la clase mayoritaria.
- Columnas de la tabla: id, texto (la pregunta X), etiqueta (la respuesta conocida y) y canal.

Ruido Realista de Bolivia Incluido a Propósito

- Montos locales: 'Nos cobraron 350 bolivianos (bs. 350) por un plato frío'.
- Enlaces web y menciones: 'Pésima atención en @hotel_central, reclamo en https://ejemplo.com'.
- Tildes, mayúsculas y signos repetidos: '¡¡¡PÉSIMO servicio, habitación suciá!!!'.
- 36 casos difíciles únicos: Oraciones con contraste ('pero', 'aunque') e ironía para evaluar errores.
- Hallazgo del EDA: Las 20 palabras más frecuentes en crudo son conectores ('de', 'la', 'el') -> exige limpieza NLP.

Explica: Erlan Garcia (Rol 1)

Normalización con Expresiones Regulares, Unidecode y Tokenización

Paso 1 y 2: Normalización y unidecode()

- 1. Minúsculas (.lower()): Para que 'Excelente' y 'excelente' sean la misma palabra.
- 2. Reemplazo semántico con Regex (re.sub): No borramos precios ni enlaces; los convertimos en tokens:
- • 'https://...' -> palabra 'url' | '@usuario' -> palabra 'usuario'
- • '500 bolivianos' o 'bs 200' -> palabra 'monto' | dígitos -> palabra 'num'
- 3. Quitar tildes con unidecode (visto en clase): 'pésimo' y 'pesimo' se unifican en 'pesimo' para no duplicar columnas.

Paso 3: Tokenización en Español con NLTK

- Limpiamos signos de puntuación y emojis dejando solo letras de la 'a' a la 'z'.
- Usamos word_tokenize(texto, language='spanish') de NLTK para dividir la oración en una lista de palabras (tokens).
- Ejemplo real de transformación en nuestro código:
- • Antes: '¡Nos cobraron 450 bs en @restaurante_sur! Pésimo servicio.'
- • Normalizado: 'nos cobraron monto en usuario pesimo servicio'
- • Tokens: ['nos', 'cobraron', 'monto', 'en', 'usuario', 'pesimo', 'servicio']

Explica: Erlan Garcia (Rol 1)

Filtrado de Stop Words: ¿Por qué protegimos la palabra 'NO'?

¿Para qué sirven las Stop Words (Palabras Vacías)?

- Como vimos en clase, artículos y preposiciones ('el', 'la', 'de', 'que', 'en') aparecen en todas las reseñas.
- Si no las quitamos, llenan la matriz de columnas inútiles con ceros y consumen memoria sin aportar sentimiento.
- Cargamos stopwords.words('spanish') de NLTK y las pasamos por unidecode() para que coincidan sin tildes.

El Problema de NLTK Estándar y Nuestra Solución

- En la lista por defecto de NLTK en español, las palabras 'no', 'ni', 'sin', 'nada' y 'muy' están marcadas para borrarse.
- ¿Qué pasa si borramos 'no' en análisis de sentimiento?
- • Reseña real: 'No recomiendo para nada este hotel'
- • Si usamos NLTK normal: queda 'recomendar hotel' -> ¡El modelo predice POSITIVO (Falso Positivo)!
- • Con nuestra lista protegida: queda 'no recomendar nada hotel' -> Predice NEGATIVO correctamente.
- Protegimos 12 palabras clave: no, ni, nunca, jamas, sin, tampoco, nada, muy, mas, poco, peor, mejor.

Explica: María Montaño (Rol 2)

Stemming vs. Lematización y Unigramas vs. Bigramas en TF-IDF

Palabra Conjugada	Stemming (Snowball)	Lematización (Dominio)
tardaron / tardó	tard (raíz truncada)	tardar (verbo base)
recomendado / recomiendo	recomend	recomendar
habitaciones	habit (choca con hábito)	habitacion
F1-Macro (Reg. Logística)	**0.9917**	**0.9917 (Elegido por legibilidad)**

Conclusión Experimento 1: ¿Por qué Lematización?

- SnowballStemmer recorta sufijos a ciegas dejando raíces cortadas ('recomend', 'pesim').
- Nuestra lematización convierte verbos y plurales a su palabra real de diccionario, facilitando explicar qué palabras pesan más en el modelo.

Experimento 2: De Texto a Números con TF-IDF y Bigramas (1,2)

- El modelo matemático no lee letras; necesita una tabla numérica (TF-IDF da más peso a palabras decisivas y menos a las repetidas).
- Comparamos Unigramas (1,1) contra Unigramas + Bigramas (1,2):
- • Con (1,1) el modelo ve 'no' y 'recomendar' por separado.
- • Con (1,2) aprende el par unido: 'no recomendar' (-1.72), 'habitacion sucio' (-1.73), 'sin duda' (+1.28).
- Parámetros documentados en TfidfVectorizer:
- • max_features=2500 columnas | min_df=2 (ignora errores de tipeo que salen 1 sola vez) | max_df=0.95.

Explica: María Montaño (Rol 2)

Partición 80/20 (fit vs. transform) y Naive Bayes vs. Regresión Logística

Regla de Clase para Separar Datos: train_test_split (80% Entrenamiento / 20% Prueba con stratify=y)

- Entrenamiento (960 reseñas = 80%): Aquí usamos fit_transform() para aprender el vocabulario y entrenar los modelos con validación cruzada de 5 pliegues (cv=5).
- Prueba Intocable (240 reseñas = 20%): Aquí SOLO usamos transform() y predict(). Jamás se usa para elegir hiperparámetros para evitar que el modelo memorice el examen.

Modelo 1: Naive Bayes Multinomial (MultinomialNB)

- Visto en clase de detección de spam/fraude: calcula la probabilidad condicional de cada clase contando la frecuencia de cada palabra.
- Usa Suavizado de Laplace (alpha = 0.1): suma un pequeño valor para que una palabra nueva nunca multiplique la probabilidad por cero.
- Muy rápido y efectivo cuando las palabras actúan como señales independientes.

Modelo 2: Regresión Logística (LogisticRegression)

- Visto en clase: asigna un peso positivo o negativo (coef_) a cada palabra o bigrama del TF-IDF.
- Pasa la suma total por la Curva Sigmoide (en forma de S) que convierte cualquier número en una probabilidad entre 0 y 1.
- Usa el umbral de decisión en 0.5 (>= 0.5 es Positivo, < 0.5 es Negativo) y regularización C = 5.0.

Explica: María Montaño (Rol 2)

La Escalera de Modelos: 'La justificación es una tabla, no una opinión'

Escalón	Modelo Evaluado	Mejores Hiperparámetros	F1 Validación (CV=5)	F1 Prueba (20%)	Exactitud	Decisión
Escalón 0	Línea Base (DummyClassifier)	Por defecto	0.3333	0.3333	50.00%	Rival mínimo a vencer
Escalón 1	Naive Bayes Multinomial	alpha=0.1, ngram=(1,2)	0.9927	0.9958	99.58%	Modelo alternativo en vivo
Escalón 2	**Regresión Logística**	**C=5.0, ngram=(1,2)**	**0.9958 (Ganador CV)**	**0.9917**	**99.17%**	**MODELO EN PRODUCCIÓN**

¿Por qué elegimos Regresión Logística como Modelo Principal en Producción?

- 1. Elección honesta sin tocar el conjunto de prueba: En la validación cruzada sobre los 960 datos de entrenamiento, Regresión Logística obtuvo el mayor F1-Macro (0.9958 vs 0.9927).
- 2. Como explicó el docente en clase, sacar 100% suele indicar datos repetidos; un ~99.2% en datos de prueba demuestra generalización real.
- 3. Además, guardamos ambos modelos (.joblib) para que en la aplicación web podamos comparar las dos predicciones al mismo tiempo.

Explica: Luis Miguel Nunez (Rol 4)

Matrices de Confusión (240 Pruebas) y Palabras con Mayor Peso

Matriz Reg. Logística (N=240)	Predicho: NEGATIVO	Predicho: POSITIVO
Real: NEGATIVO (120 quejas)	**120 (Verdadero Negativo)**	**0 (Falso Positivo)**
Real: POSITIVO (120 elogios)	2 (Falso Negativo)	118 (Verdadero Positivo)

Interpretación de la Matriz de Confusión

- De las 240 reseñas del examen (20%), Regresión Logística acertó 238 y Naive Bayes acertó 239.
- ¡CERO Falsos Positivos (FP = 0) en ambos modelos!: De las 120 quejas reales, detectaron las 120. Ningún reclamo de un cliente se escapó como si fuera positivo.

¿Qué palabras aprendió el modelo? (Coeficientes coef_)

- Palabras que más empujan hacia POSITIVO (Peso > 0):
- • 'delicioso' (+3.02) | 'super' (+1.94) | 'impecable' (+1.90)
- • 'encantar' (+1.89) | 'excelente' (+1.58) | 'sin duda' (+1.28)
- Palabras y bigramas que más empujan hacia NEGATIVO (Peso < 0):
- • 'no' (-3.64) | 'pesimo' (-3.01) | 'sucio' (-2.72)
- • 'frio' (-2.55) | 'horrible' (-2.42)| 'no recomendar' (-1.72)
- Demuestra por qué conservar 'no' y usar bigramas fue decisivo.

Explica: Luis Miguel Nunez (Rol 4)

¿En qué casos falla el modelo y cuándo discrepan Logística vs. Naive Bayes?

Radiografía de los 2 Únicos Errores en Prueba

- Caso real donde falló Regresión Logística:
- 'Al principio NO aparecía nuestra reserva en https://hotel-ejemplo.com, PERO el gerente lo resolvió rápido y fue EXCELENTE.'
- Etiqueta Real: POSITIVO | Predicción Logística: NEGATIVO (54.2%).
- ¿Por qué se equivocó?: Es una oración mixta con conector adversativo ('pero'). Como el token 'no' tiene el peso negativo más fuerte (-3.64), superó por estrecho margen al peso positivo de 'excelente' (+1.58).

Comparación en Vivo: ¿Por qué aquí Naive Bayes sí acierta?

- Sobre esa misma frase difícil, nuestros dos modelos opinan distinto:
- • Regresión Logística -> NEGATIVO (54.2% confianza).
- • Naive Bayes Multinomial -> POSITIVO (86.5% confianza).
- Explicación matemática: En Naive Bayes, la suma de probabilidades logarítmicas de 'resolver' + 'rapido' + 'excelente' supera al token 'no'.
- En nuestra interfaz web, este caso activa la alerta visual: 'Los modelos discrepan'.

Explica: Luis Miguel Nunez (Rol 4)

Ficha 'Mi Escalera': Los 9 Casilleros de Nuestro Proyecto

Casillero (1 al 3)	Casillero (4 al 6)	Casillero (7 al 9)
1. EL DATO: Texto en español, 1.200 filas balanceadas (600 pos / 600 neg), 4 columnas.	4. LÍNEA BASE: DummyClassifier con F1-Macro = 0.3333 (Exactitud = 50.0%).	7. MÉTRICA PRIMARIA: F1-Macro (no reportamos Exactitud sola para cuidar falsos positivos y negativos).
2. PREGUNTA Y TAREA: ¿Cliente satisfecho o insatisfecho? -> Clasificación binaria supervisada.	5. MODELO v1 (Escalón 1): TF-IDF + MultinomialNB (alpha=0.1) -> F1 Prueba = 0.9958.	8. PARTICIÓN: 80% entreno (960) / 20% prueba (240) estratificado + 5-Fold CV en entreno.
3. RESTRICCIÓN PRINCIPAL: Explicabilidad (ver pesos), baja latencia (< 5 ms) y costo cero (sin GPU).	6. MODELO v2 (Producción): TF-IDF + Regresión Logística (C=5.0) -> Mejor F1 en CV (0.9958).	9. ¿POR QUÉ NO DEEP LEARNING / LLM?: Con 1.200 textos acotados, TF-IDF + Logística da >99% F1 offline en 2 ms sin GPU.

Explica: Ronny Choque (Rol 3)

Empaquetado en .joblib, Validación y 10 Pruebas Automatizadas

El mismo Pipeline entrena y responde en la API

- Con un solo comando (python src/models/entrenar.py) se ejecuta todo desde los datos crudos hasta guardar los modelos.
- Empaquetamos en modelo_sentimiento.joblib las 3 etapas unidas:
- 1. FunctionTransformer(limpiar_lote_textos)
- 2. TfidfVectorizer (con las 2.500 columnas aprendidas)
- 3. Clasificador (Regresión Logística / Naive Bayes)
- Así el servidor FastAPI (api/main.py) solo hace joblib.load() y jamás reimplementa ni altera el preprocesamiento.

Guardarraíles y 10 Pruebas en Pytest (100% Verde)

- Validación de entradas (api/esquemas.py): Rechaza textos vacíos, de 1 sola palabra o solo números/símbolos ('123 !!!') devolviendo mensaje claro en español.
- Suite tests/test_sistema.py (10 passed):
- • Prueba unitaria de normalización (URLs, montos en Bs y tildes).
- • Prueba unitaria que verifica que la palabra 'no' NUNCA sea eliminada.
- • Prueba de Stemming vs. Lematización y de todos los endpoints (/health, /metrics, /predict y lotes).

Explica: Todo el Equipo

Radiografía Visual de los 6 Pasos de NLP y Demostración en Vivo

¿Qué muestra nuestra Interfaz Web en cada predicción?

- En vez de dar solo una respuesta cerrada, muestra las 6 etapas del pipeline de NLP en vivo:
- 1. Texto Original ingresado.
- 2. Texto Normalizado (resaltando tokens 'url' y 'monto').
- 3. Tokens separados palabra por palabra.
- 4. Filtrado de Stop Words (destacando en ámbar las negaciones protegidas como 'no' y en gris las eliminadas).
- 5. Raíces de SnowballStemmer vs. 6. Lemas finales que entran a TF-IDF.

Pasamos a la Demostración en Vivo (http://127.0.0.1:8000)

- Prueba 1: Reseña positiva con monto en bolivianos.
- Prueba 2: Reseña negativa con negación 'NO' ('No recomiendo nada este lugar...').
- Prueba 3: Caso ambiguo con conector 'pero' donde Regresión Logística y Naive Bayes discrepan en vivo.
- Prueba 4: Clasificación por lote de 5 reseñas simultáneas y revisión del panel de matrices de confusión.